



ORACLE

Academy

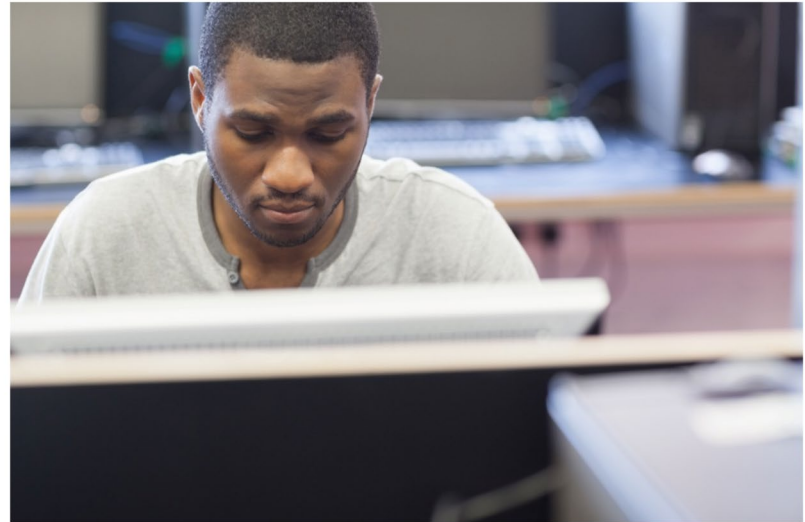


Java Fundamentals

4-2

Classes de pilote et d'objet

ORACLE
Academy



Objectifs

- Voici les objectifs de cette leçon :
 - Décrire l'aspect général d'un programme Java
 - Décrire la différence entre une classe d'objet et une classe de pilote
 - Accéder à au moins deux API de classe Java
 - Expliquer les mots-clés Java et donner des exemples
 - Créer une classe d'objet
 - Créer une classe de pilote



Forme générale de programme Java

- Il existe deux formes générales de classes Java :
 - Classes de pilote
 - Classes d'objet

Classes de pilote

- Caractéristiques des classes de pilote :
 - Contiennent une méthode principale
 - Une méthode principale est nécessaire pour exécuter un programme Java
 - La méthode principale peut inclure :
 - Des instances d'objets provenant d'une classe d'objet
 - Variables
 - Des boucles, des instructions conditionnelles (if-else)
 - Une autre logique de programmation
 - Peuvent également contenir d'autres méthodes statiques

Classes d'objet

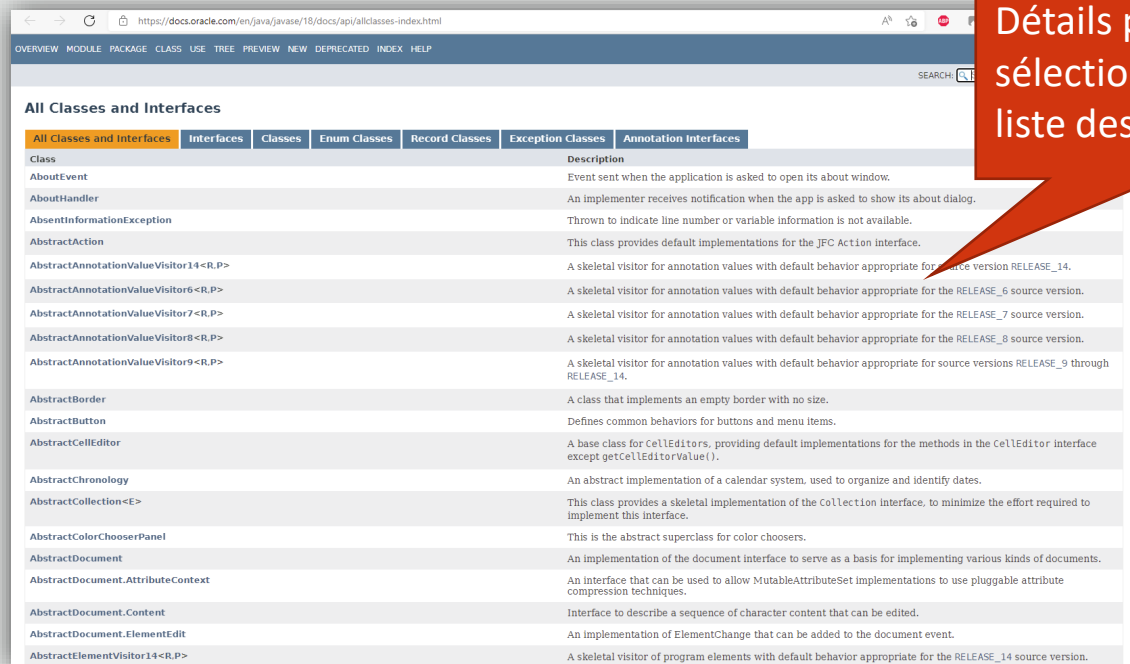
- Caractéristiques des classes d'objet :
 - Sont des classes qui définissent les objets à utiliser dans une classe de pilote
 - Peuvent être trouvées dans l'API Java ou créées par vous
 - Exemples : String, BankAccount, Student, Rectangle

L'API Java est une bibliothèque de packages et de classes d'objet qui sont déjà écrits et qui sont disponibles pour utilisation dans vos programmes



API Java

- Vous pouvez trouver la documentation relative à l'API Java ici :
 - <https://docs.oracle.com/en/java/javase/18/docs/api/allclasses-index.html>



The screenshot shows the Oracle Java API documentation website for Java SE 18. The page title is "All Classes and Interfaces". There are several tabs: "All Classes and Interfaces" (selected), "Interfaces", "Classes", "Enum Classes", "Record Classes", "Exception Classes", and "Annotation Interfaces". Below the tabs is a table with two columns: "Class" and "Description". The table lists various classes and interfaces, including AbstractEvent, AboutHandler, AbsentInformationException, AbstractAction, and several AbstractAnnotationValueVisitor classes for different source versions. A red callout bubble with white text points to the "All Classes and Interfaces" tab, stating: "Détails pour la classe sélectionnée dans la liste des classes".

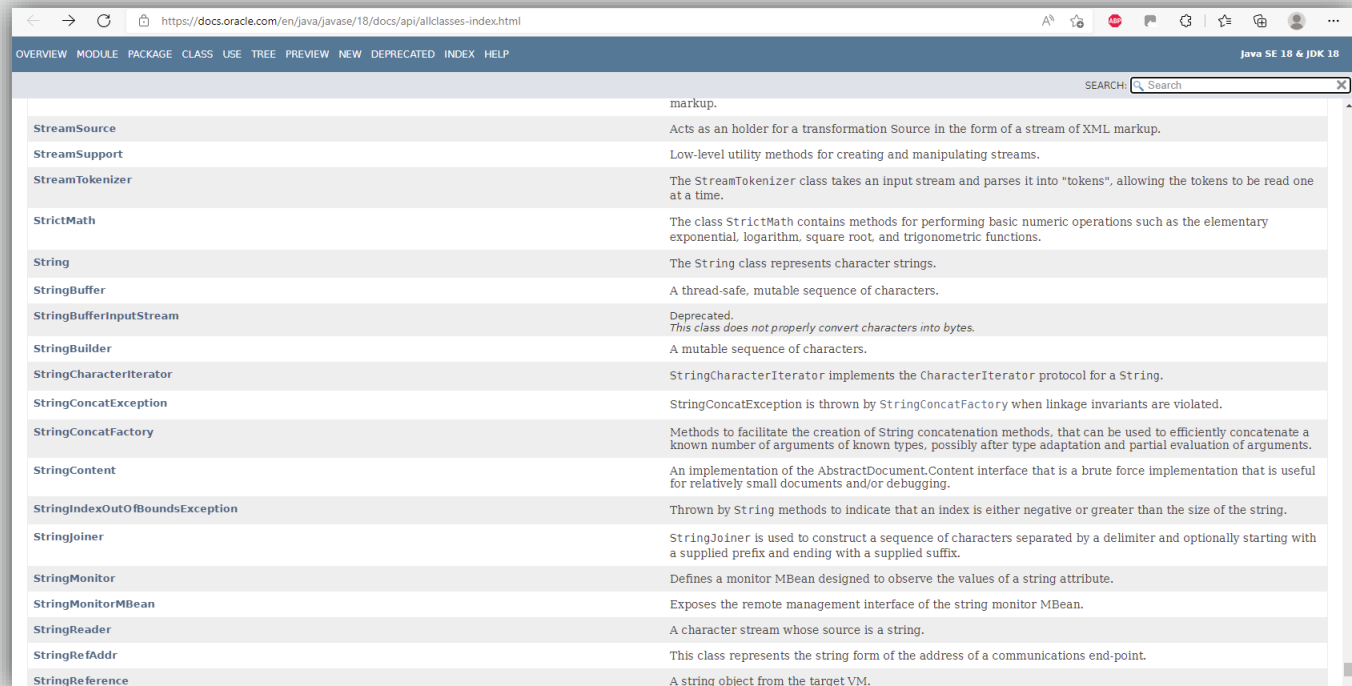
Class	Description
AbstractEvent	Event sent when the application is asked to open its about window.
AboutHandler	An implementer receives notification when the app is asked to show its about dialog.
AbsentInformationException	Thrown to indicate line number or variable information is not available.
AbstractAction	This class provides default implementations for the JFC Action interface.
AbstractAnnotationValueVisitor14<R,P>	A skeletal visitor for annotation values with default behavior appropriate for source version RELEASE_14.
AbstractAnnotationValueVisitor6<R,P>	A skeletal visitor for annotation values with default behavior appropriate for the RELEASE_6 source version.
AbstractAnnotationValueVisitor7<R,P>	A skeletal visitor for annotation values with default behavior appropriate for the RELEASE_7 source version.
AbstractAnnotationValueVisitor8<R,P>	A skeletal visitor for annotation values with default behavior appropriate for the RELEASE_8 source version.
AbstractAnnotationValueVisitor9<R,P>	A skeletal visitor for annotation values with default behavior appropriate for source versions RELEASE_9 through RELEASE_14.
AbstractBorder	A class that implements an empty border with no size.
AbstractButton	Defines common behaviors for buttons and menu items.
AbstractCellEditor	A base class for CellEditors, providing default implementations for the methods in the CellEditor interface except getCellEditorValue().
AbstractChronology	An abstract implementation of a calendar system, used to organize and identify dates.
AbstractCollection<E>	This class provides a skeletal implementation of the Collection interface, to minimize the effort required to implement this interface.
AbstractColorChooserPanel	This is the abstract superclass for color choosers.
AbstractDocument	An implementation of the document interface to serve as a basis for implementing various kinds of documents.
AbstractDocument.AttributeContext	An interface that can be used to allow MutableAttributeSet implementations to use pluggable attribute compression techniques.
AbstractDocument.Content	Interface to describe a sequence of character content that can be edited.
AbstractDocument.ElementEdit	An implementation of ElementChange that can be added to the document event.
AbstractElementVisitor14<R,P>	A skeletal visitor of program elements with default behavior appropriate for the RELEASE_14 source version.

Détails pour la classe sélectionnée dans la liste des classes

API Java : classe String

- Faites défiler la liste des classes jusqu'à ce que String apparaisse, puis cliquez sur le lien
- Vous devriez voir ce qui suit :

Faire défiler
cette liste
jusqu'à
String

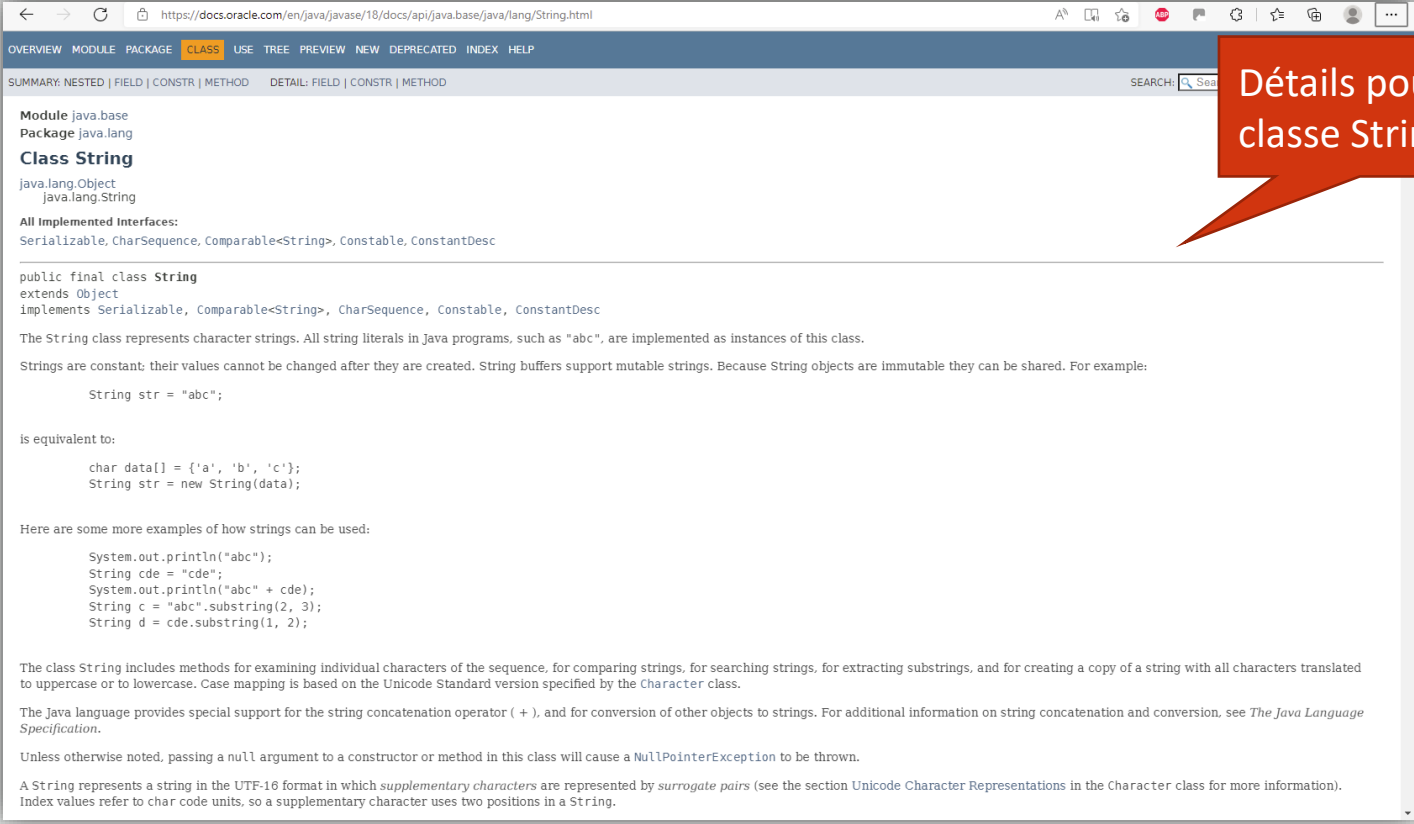


The screenshot shows the Oracle Java API documentation page for the String class. The page is titled "String" and lists various subclasses and related classes. The table below is a representation of the content shown in the screenshot.

Class Name	Description
StreamSource	Acts as an holder for a transformation Source in the form of a stream of XML markup.
StreamSupport	Low-level utility methods for creating and manipulating streams.
StreamTokenizer	The StreamTokenizer class takes an input stream and parses it into "tokens", allowing the tokens to be read one at a time.
StrictMath	The class StrictMath contains methods for performing basic numeric operations such as the elementary exponential, logarithm, square root, and trigonometric functions.
String	The String class represents character strings.
StringBuffer	A thread-safe, mutable sequence of characters.
StringBufferInputStream	Deprecated. This class does not properly convert characters into bytes.
StringBuilder	A mutable sequence of characters.
StringCharacterIterator	StringCharacterIterator implements the CharacterIterator protocol for a String.
StringConcatException	StringConcatException is thrown by StringConcatFactory when linkage invariants are violated.
StringConcatFactory	Methods to facilitate the creation of String concatenation methods, that can be used to efficiently concatenate a known number of arguments of known types, possibly after type adaptation and partial evaluation of arguments.
StringContent	An implementation of the AbstractDocument.Content interface that is a brute force implementation that is useful for relatively small documents and/or debugging.
StringIndexOutOfBoundsException	Thrown by String methods to indicate that an index is either negative or greater than the size of the string.
StringJoiner	StringJoiner is used to construct a sequence of characters separated by a delimiter and optionally starting with a supplied prefix and ending with a supplied suffix.
StringMonitor	Defines a monitor MBean designed to observe the values of a string attribute.
StringMonitorMBean	Exposes the remote management interface of the string monitor MBean.
StringReader	A character stream whose source is a string.
StringRefAddr	This class represents the string form of the address of a communications end-point.
StringReference	A string object from the target VM.

API Java : étudier la classe String

- Etudiez et abordez les constructeurs et les méthodes



The screenshot shows the Oracle Java API documentation for the `String` class. The browser address bar shows the URL `https://docs.oracle.com/en/java/javase/18/docs/api/java.base/java/lang/String.html`. The page has a navigation bar with tabs for OVERVIEW, MODULE, PACKAGE, CLASS (selected), USE, TREE, PREVIEW, NEW, DEPRECATED, INDEX, and HELP. Below the navigation bar, there are links for SUMMARY, NESTED, FIELD, CONSTR, METHOD, and DETAIL. The main content area displays the following information:

- Module:** java.base
- Package:** java.lang
- Class:** String
- Interfaces:** java.lang.Object, java.lang.String
- All implemented interfaces:** Serializable, CharSequence, Comparable<String>, Constable, ConstantDesc

The class definition is shown as:

```
public final class String
extends Object
implements Serializable, Comparable<String>, CharSequence, Constable, ConstantDesc
```

The documentation explains that the `String` class represents character strings and that string literals in Java programs are implemented as instances of this class. It also notes that strings are constant and their values cannot be changed after they are created.

Example code snippets are provided, such as:

```
String str = "abc";
```

and

```
char data[] = {'a', 'b', 'c'};
String str = new String(data);
```

The page also includes examples of how strings can be used, such as:

```
System.out.println("abc");
String cde = "cde";
System.out.println("abc" + cde);
String c = "abc".substring(2, 3);
String d = cde.substring(1, 2);
```

The documentation further describes the methods provided by the `String` class for examining individual characters, comparing strings, searching strings, extracting substrings, and creating a copy of a string with all characters translated to uppercase or to lowercase. It also mentions the special support for the string concatenation operator (`+`) and for conversion of other objects to strings.

Détails pour la
classe String

API Java : classe String

- Nous allons utiliser la classe String dans des programmes
- Constructeur le plus courant pour cette classe :
 - `String(String original)`
- Méthodes courantes :

Méthode	Description
<code>charAt(int index)</code>	Renvoie la valeur char à l'index indiqué
<code>length()</code>	Renvoie la longueur de la chaîne
<code>substring(int beginIndex)</code>	Renvoie une nouvelle chaîne qui est une sous-chaîne de cette chaîne

Classe d'objet simple créée par le programmeur : Student

- Une classe Java est utilisée pour stocker ou représenter des données pour l'objet que la classe représente
- Il existe de nombreuses classes déjà disponibles à partir de l'API Java, mais vous devrez probablement en créer bien d'autres
- Par exemple, nous pouvons créer un modèle, ou représentation par programmation, d'étudiant
- Les informations dont nous pourrions avoir besoin pour un étudiant sont son ID, son nom et son GPA
- Dans cette leçon, nous allons créer une classe d'objet appelée Student, puis une classe de pilote appelée StudentTester

Classe d'objet simple créée par le programmeur : Student

- Toutes les classes Java sont créées dans un fichier texte du même nom que la classe
- Ces fichiers possèdent également une extension .java
- Dans cette leçon, nous allons créer une classe Student et une classe StudentTester dans votre IDE Java



Syntaxe de création d'une classe d'objet simple créée par le programmeur

- Voici un exemple de syntaxe pour la création d'une classe d'objet créée par le programmeur
- Mots-clés Java :
 - package (facultatif)
 - import (facultatif)
 - public class

```
package <package_name>;  
import <other_packages>;  
public class ClassName  
{  
    <variables (also known as fields)>;  
    <constructor method(s)>;  
    <other methods>;  
}
```

Termes clés



Terme	Définition
package (mot-clé)	• Définit l'emplacement de cette classe par rapport aux autres classes et offre un niveau de contrôle d'accès • Utilisez des modificateurs d'accès (tels que public et private) pour contrôler l'accès
import (mot-clé)	• Définit d'autres classes ou groupes de classes que vous utilisez dans votre classe • L'instruction import fournit les informations du compilateur qui identifient les classes externes utilisées dans la classe actuelle
class (mot-clé)	• Précède le nom de la classe • Le nom de la classe et le nom du fichier doivent correspondre lorsque la classe est déclarée publique (ce qui est une bonne pratique) • Cependant, le mot-clé public devant le mot-clé class est un modificateur et n'est pas requis

Termes clés



Terme	Définition
variables de classe ou champs d'instance (souvent appelés champs)	Variables ou données associées aux programmes (telles que les entiers, les chaînes, les tableaux et les références à d'autres objets)
Constructeurs	Méthodes appelées lors de la création (instanciation) d'un objet (représentation en mémoire d'une classe Java)
Méthodes	- Méthodes qui peuvent être exécutées sur un objet - Elles sont également appelées méthodes d'instance - Les méthodes peuvent renvoyer ou modifier les valeurs de variable d'un objet (parfois appelées fonctions)

Mots-clés Java

- Tous les programmes Java utilisent des mots clés Java
- Exemples : class, public, String, int, for, while et double
- La couleur de police pour les mots-clés changera dans votre IDE Java
- Liste des mots-clés Java :
 - <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/keywords.html>

Un mot-clé Java est un mot ayant une fonction spéciale dans le langage Java et ne pouvant pas être utilisé en tant que nom de classe, de méthode ou de variable



Conventions de dénomination du programme Java



- Conventions de dénomination pour un programme Java :
 - Le nom de package facultatif est défini avant une instruction import en lowerCamelCase
 - Les instructions import facultatives sont définies sous le nom du package
 - Le nom de classe est un nom en UpperCamelCase

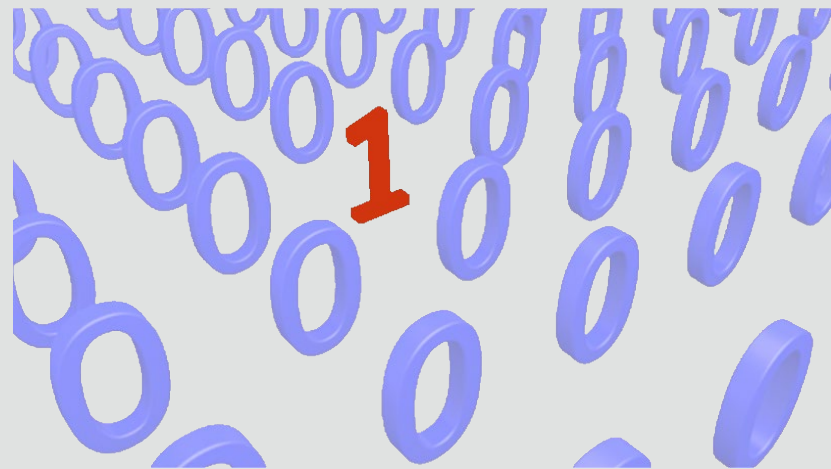


CamelCase

- La notation CamelCase est la pratique consistant à lier des mots commençant par une majuscule et sans espace
- La notation lowerCamelCase lie des mots commençant par une majuscule, sauf le premier mot qui commence par une minuscule
- Par exemple :
 - thisIsLowerCamelCase
- La notation UpperCamelCase lie des mots commençant par une majuscule, y compris le premier mot
- Par exemple :
 - ThisIsUpperCamelCase

Conventions de dénomination supplémentaires pour un programme Java

- Conventions de dénomination supplémentaires pour un programme Java :
 - Les noms de variable sont courts, mais significatifs en lowerCamelCase
 - Les noms de constante sont déclarés en lettres majuscules avec le modificateur final
 - Le nom des constructeurs est identique au nom de classe
 - Les méthodes sont des verbes en lowerCamelCase



Exemple de convention de dénomination

- Les lignes directrices pour une classe d'objet créée par le programmeur sont marquées dans cet exemple pour Student
 - Créez ce fichier dans votre IDE Java // est le symbole des commentaires

```
package com.example.domain; // Déclaration de package
import java.util.Scanner;    // Instruction import pour d'autres packages
public class Student {      // Déclaration de classe pour ce fichier
    private int studentId;   // Déclarations de variable pour cette cl
    private String name;
    private String ssn;
    private double gpa;
    public final int SCHCODE = 34958 // Déclaration de constante
    public Student() {        // Constructeur
    } //fin Constructeur
    public int getStudentId() { // Méthode d'accès
        return studentId;
    } //fin methode studentId
    public void setStudentId(int x) { // Modificateur
        studentId = x;
    } //fin methode setStudentId
} //fin classe Student
```

Classe d'objet simple créée par le programmeur : Student

- Le mot-clé `package` facultatif est utilisé dans Java pour regrouper des classes
 - Un package est implémenté en tant que dossier
 - Comme un dossier, il fournit un espace de noms à une classe
 - Il est recommandé de toujours déclarer un package dans la partie supérieure de la classe

```
package com.example.domain;
```

Classe d'objet simple créée par le programmeur : Student

- Dans l'exemple ci-dessous, une classe Student et une classe Teacher peuvent se trouver dans un dossier sous le nom de domaine pour chaque développeur
- Si une société appelée Acme emploie les développeurs nommés Smith et Jones, les packages peuvent être :
 - package com.acme.smith
 - package com.acme.jones
- Le chemin du fichier de Jones est indiqué ci-dessous

Vue de l'espace de noms pour Jones :

- com.acme.jones
 - Student
 - Teacher

Arborescence pour Jones :

```
+com
|_+acme
|   |_+jones
|       |_+Student.java
|       |_+Teacher.java
```

Mot-clé import

- Le mot-clé import est utilisé pour identifier les packages ou classes d'objet que vous souhaitez utiliser dans votre classe
- Vous pouvez importer une seule classe ou un package entier
- Vous pouvez inclure plusieurs instructions import
- Les instructions import suivent la déclaration de package et précèdent la déclaration de classe

```
import java.util.Scanner;
```

Instructions import

- Aucune instruction import n'est obligatoire et, par défaut, votre classe importe toujours `java.lang` à partir de l'API
 - <https://docs.oracle.com/en/java/javase/18/docs/api/java.base/java/lang/Package.html>
- Vous n'avez pas à importer les classes qui sont dans le même package que l'instruction import

Exemples d'instructions import

- Voici d'autres exemples d'instructions d'import :

```
import java.util.Date;    //Importer la classe individuelle.
import java.util.*;       //Importer le package entier.
import java.util.Date;    //Importer à l'aide de plusieurs
                          //instructions.
import java.util.Calendar;
```

Variables pour la classe Student

- En plus des instructions package et import, la classe Student contient des variables pour l'ID étudiant, le nom, le numéro de sécurité sociale, le GPA, ainsi que le code de l'établissement
- Pour cela, une classe avec des variables de classe et un constructeur doit être définie
- En outre, des méthodes pouvant accéder aux variables et les modifier sont ajoutées



Classe Student

```
package com.example.domain;
```

```
public class Student {
```

```
    private int studentId;
```

```
    private String name;
```

```
    private String ssn;
```

```
    private double gpa;
```

```
    public final int SCHCODE = 34958;
```

```
    public Student() {
```

```
    } //fin constructeur
```

```
    public int getStudentId() {
```

```
        return studentId;
```

```
    } //fin méthode getStudentId
```

```
    public void setStudentId(int x) {
```

```
        studentId = x;
```

```
    } //fin méthode setStudentId
```

```
} //fin classe Student
```

Déclaration de classe

Champs/Variables

Constructeur

Méthode

Modificateur d'accès

Conventions pour la déclaration de classe

- Conventions de déclaration de classe :
 - Le nom doit commencer par un caractère et peut contenir des nombres, _ ou \$
 - Utilisez la notation UpperCamelCase si le nom est composé de plusieurs mots
 - Pour une classe d'objet simple créée par le programmeur, le modificateur d'accès doit être public (tous les modificateurs d'accès sont publics, privés ou protégés)

```
public class Student{}
```

L'ensemble du code pour cette classe doit être contenu dans des accolades { }

Conventions pour les variables de classe

- Conventions des variables de classe :
 - Les variables de classe doivent être déclarées avec le modificateur d'accès privé pour protéger les données
 - Les variables de classe sont nommées à l'aide de la notation lowerCamelCase
 - Exception : les constantes (valeurs qui ne changent pas) qui doivent être nommées à l'aide de la notation UpperCamelCase et déclarées publiques pour permettre aux programmes de pilote d'accéder à la valeur

Exemples de déclaration de variables de classe

- Exemples de déclaration de variables de classe :
 - `private int length;`
 - `private int width;`
 - `private double area; //constante`
 - `public final double SCALE = 0.25;`
 - `private String name;`



Méthodes de constructeur

- Une méthode de constructeur est unique dans Java pour les raisons suivantes :
 - La méthode crée une instance de la classe
 - Les constructeurs possèdent toujours le même nom que la classe et ne déclarent aucun type de retour

```
public Student()  
{  
  
} //fin constructeur
```

L'ensemble du code pour cette méthode doit être contenu dans des accolades { }

Méthodes de constructeur

- Avec les méthodes de constructeur :
 - Vous pouvez déclarer plus d'un constructeur dans une classe
 - Vous n'êtes pas obligé de déclarer un constructeur, Java vous fournit un constructeur par défaut (vide)
 - Si vous déclarez des constructeurs, Java ne fournit aucun constructeur par défaut

```
public Student()  
{  
  
}  
//fin constructeur
```

L'ensemble du code pour cette méthode doit être contenu dans des accolades { }

Constructeurs sans paramètre

- Si vous créez un constructeur sans paramètre (la parenthèse est vide), vous pouvez laisser le contenu du constructeur (entre les accolades { et }) vide
- Il est appelé constructeur par défaut et est identique au constructeur fourni par Java si vous n'en déclarez pas
- Ce constructeur initialise les variables de classe numériques sur zéro et les variables d'objet (telles que des chaînes) sur NULL

```
public Student()  
{  
}  
} //fin constructeur
```

Aucun paramètre

Constructeurs sans paramètre

- Si vous créez un constructeur sans paramètre (la parenthèse est vide), vous pouvez également initialiser les variables entre les accolades { et }
 - Il s'agit aussi d'un constructeur par défaut
 - Ce constructeur initialise les variables de classe numériques sur zéro et les variables d'objet (telles que des chaînes) sur NULL
 - Il a les mêmes résultats que sur la diapositive précédente, mais les valeurs sont plus évidentes

```
public Student(){  
    studentId = 0;  
    name = "";  
    ssn = "";  
    gpa = 0.0;  
} //fin constructeur
```

Aucun paramètre

Constructeurs avec paramètres

- Si vous créez un constructeur avec des paramètres (la parenthèse n'est PAS vide), vous initialisez également les variables entre les accolades { et }
- Ce constructeur initialise les variables de classe avec les valeurs envoyées à partir de la classe de pilote principale

```
public Student(int x, String n, String s, double g){  
    studentId = x;  
    name = n;  
    ssn = s;  
    gpa = g;  
} //fin constructeur
```

Exemple de constructeur par défaut

- La méthode de constructeur dans cet exemple est un constructeur par défaut qui crée une instance de Student

```
package com.example.domain;
public class Student
{
    private int studentId;
    private String name;
    private String ssn;
    private double gpa;
    public final int SCHCODE = 34958;

    public Student() {
    } //fin constructeur
    public int getStudentId() {
        return studentId;
    } //fin methode getStudentId
} //fin classe Student
```

Constructeur

Exemple de constructeur par défaut

- L'exemple Student sur la diapositive précédente illustre un constructeur sans argument simple
- La valeur renvoyée par le constructeur est une référence à un objet Java du type créé
- Les constructeurs peuvent également comporter des paramètres

Constructeurs

- Un constructeur est une méthode qui crée un objet
- Dans Java, les constructeurs sont des méthodes portant le même nom que leur classe permettant de créer une instance d'un objet
- Les constructeurs sont appelés à l'aide du mot-clé new
- Exemple de code qui pourrait être utilisé dans une classe de pilote pour créer un objet à partir du constructeur Student :

```
Student stu = new Student();
```

Méthode principale

- Pour exécuter un programme Java, vous devez définir une méthode principale dans une classe de pilote
- La méthode principale est automatiquement appelée lorsque la classe est appelée
- N'oubliez-pas d'attribuer au fichier le même nom que la classe

Classe de pilote simple créée par le programmeur : StudentTester

- Pour les exemples de ce cours, nous utiliserons souvent le nom de la classe d'objet suivi du mot "Tester"
- Voici un exemple de classe de pilote Java simple nommée StudentTester avec une méthode principale

```
public class StudentTester
{
    public static void main(String args[])
    {
        }//fin méthode main
} //end classe StudentTester
```

Classe de pilote simple créée par le programmeur : StudentTester - Exemple 1

- Dans cet exemple, une instruction est ajoutée pour créer un objet Student
- Student est créé et les variables de classe sont initialisées tel que décrit précédemment sous les constructeurs par défaut

```
public class StudentTester
{
    public static void main(String args[])
    {
        Student s1 = new Student();
    } //fin méthode main
} //fin classe StudentTester
```

Classe de pilote simple créée par le programmeur : StudentTester - Exemple 2

- Dans cet exemple, l'instruction permettant de créer un objet de la classe Student est différente
- Cet objet est créé en utilisant des paramètres
- Pouvez-vous deviner l'ID de l'étudiant ? Son nom ? Son numéro de sécurité sociale ? Son GPA ?
- Ajoutez un autre étudiant en utilisant le constructeur par défaut

```
public class StudentTester
{
    public static void main(String args[])
    {
        Student s1 = new Student(123, "Mary Smith", "999-99-9999", 3.4);
    } //fin méthode main
} //fin classe StudentTester
```

Amélioration de la classe d'objet Student

- La plupart des environnements de développement intégrés modernes offrent un moyen facile de générer automatiquement les méthodes d'accès (get) et les modificateurs (set)
- Une autre méthode utile lors des tests, de la création et de la modification d'objets est la méthode toString()
- Dans les prochaines diapositives, nous développerons de nouvelles méthodes pour la classe d'objet Student et nous modifierons la classe de pilote StudentTester afin de les tester

Méthodes d'accès et modificateurs

- Il est fréquent de créer un ensemble de méthodes qui manipulent et extraient des valeurs de données de classe :
 - Méthodes d'accès (méthodes get) :
 - méthodes qui renvoient (obtiennent) la valeur de chaque variable de classe
 - Modificateurs (méthodes set) :
 - méthodes qui modifient (définissent) la valeur de chaque variable de classe

Méthodes d'accès et modificateurs pour améliorer la classe d'objet Student

- Exemples (à suivre) :
 - Ajoutez les méthodes d'accès suivantes :
 - `getStudentID()`
 - `getName()`
 - `getSSN()`
 - `getGPA()`
 - Ajoutez les modificateurs suivants :
 - `setStudentID()`
 - `setName()`
 - `setSSN()`
 - `setGPA()`
 - Ajoutez une méthode `toString()` à la classe Student afin de voir les données Student en tant que sortie

Amélioration de la classe d'objet Student

- Ajoutez les méthodes `getStudentId()` et `setStudentId()`, comme illustré ci-dessous
- Le nom d'objet de la classe (`this`) est utilisé pour différencier la variable de classe `studentId` du paramètre `studentId` transmis en tant qu'argument

```
public int getStudentId() {  
    return studentId;  
} //fin méthode getStudentId  
  
public void setStudentId(int studentId) {  
    this.studentId = studentId;  
} //fin méthode setStudentId
```

Méthodes supplémentaires pour la classe d'objet Student

- Maintenant, en utilisant l'exemple de la diapositive précédente, ajoutez les méthodes suivantes à la classe d'objet Student :
 - getName()
 - getSSN()
 - getGPA()
 - setName()
 - setSSN()
 - setGPA()

Ajouter la méthode toString() à la classe d'objet Student

- Maintenant, ajoutez la méthode toString() à la classe d'objet Student
- Vous pouvez construire des objets String afin d'afficher les informations sur chaque étudiant :

```
public String toString()
{
    String s1 = "";
    s1 = "Student Id: " + getStudentId()
        + "Student Name: " + getName()
        + "Student SSN: " + getSSN()
        + "Student GPA: " + getGPA();
    return s1;
} //fin méthode toString
```

Création de la méthode principale de classe de pilote StudentTester

- Maintenant, créez la méthode principale de classe de pilote StudentTester comme suit :

```
public class StudentTester
{
    public static void main(String args[])
    {
        Student s1 = new Student(123, "Mary Smith",
                                "999-99-9999", 3.4);
        System.out.println(s1);
        Student s2 = new Student();
        s2.setStudentId(124);
        s2.setStudentName("John Jacoby");
        s2.setSSN("123-45-6789");
        s2.setGPA(4.0);
        System.out.println(s2);
        Student s3 = new Student();
        System.out.println(s3);
    } //fin méthode main
} //fin classe StudentTester
```

Blocs de code

- Un bloc de code est défini par l'ouverture et la fermeture des accolades { }
- Lors de l'examen des blocs de code, tenez compte des points suivants :
 - Chaque déclaration de classe est contenue dans un bloc de code
 - Les déclarations de méthode, y compris la méthode principale, sont contenues dans des blocs de code
 - Les champs et les méthodes Java possèdent une portée de bloc (ou de classe)

```
public class SayHello{           //Commencer le bloc de code de la classe
    public static void main(String args[] { /*Commencer le bloc de code de la
                                           méthode*
        System.out.println("Hello Lisa");
    } //fin methode main           // Terminer le bloc de code de la méthode
} //fin classe StudentTester      // Terminer le bloc de code de la classe
```

Format des blocs de code

- Les blocs de code commencent par une accolade ouvrante **{ et se terminent par une accolade fermante }**
- Chaque fois que vous commencez un bloc de code, vous devez le terminer
- Par exemple : chaque accolade entrante **{ DOIT être complétée par une accolade fermante }**
- Où trouver les blocs de code ?
 - Classes
 - Méthodes
 - Instructions conditionnelles (instructions if, instructions switch)
 - Boucles

Terminologie

- Termes clés utilisés dans cette leçon :
 - Modificateurs d'accès
 - Blocs de code
 - Constantes
 - Constructeurs
 - Classe de pilote
 - Instructions import
 - API Java
 - Commentaires Java
 - Mots-clés Java
 - lowerCamelCase
 - Méthodes

Terminologie

- Termes clés utilisés dans cette leçon :
 - Classe d'objet
 - Packages
 - Paramètres
 - Classe créée par le programmeur
 - UpperCamelCase
 - Variables

Synthèse

- Dans cette leçon, vous avez appris comment :
 - Décrire l'aspect général d'un programme Java
 - Décrire la différence entre une classe d'objet et une classe de pilote
 - Accéder à au moins deux API de classe Java
 - Expliquer les mots-clés Java et donner des exemples
 - Créer une classe d'objet
 - Créer une classe de pilote



ORACLE
Academy